# PLAN D’ORAL

## [SLIDE 1 : TITRE]

Bonjour, aujourd’hui, je vais vous présenter mon projet Full7, une application web de gestion de collection rétro gaming.

## montrer le site

inscription -> connection -> déconnection -> connection au compte avec un full7 -> Collection (expliqué le concept du full7) ->

## [SLIDE 2 : PLAN]

Je vais commencer par présenter le contexte et les objectifs du projet,
puis le public cible et les utilisateurs.
Ensuite, je parlerai des choix techniques, de l’architecture et de la conception de la base de données.
Je détaillerai la réalisation technique, puis la sécurité et la protection des données personnelles, avant de conclure.

## [SLIDE 3 : CONTEXTE]

Pour les collectionneurs de jeux rétro, chaque jeu manquant est une pièce perdue d’un patrimoine vidéoludique. Entre les souvenirs qui s’effacent, les listes interminables et les collections qui semblent ne jamais s’achever, difficile de savoir ce que l’on possède réellement et ce qu’il reste à conquérir. Peu à peu, la passion peut se transformer en véritable quête, où chaque oubli devient un obstacle et chaque découverte, une victoire.

## [SLIDE 4 : CONTEXTE ET SOLUTION]

C’est pour répondre à ce défi que Full7 voit le jour. Une application qui permet de reprendre le contrôle de sa collection, de parcourir les consoles, de recenser ses jeux et de suivre sa progression. Car derrière chaque collection se cache une histoire, et derrière chaque jeu manquant, un objectif de plus à atteindre.

## [SLIDE 5 : OBJECTIFS]

L’objectif principal de Full7 est de permettre aux collectionneurs de jeux rétro de gérer facilement leur collection. L’application permet de créer un compte, de parcourir un catalogue de consoles et de jeux, puis d’ajouter les jeux que l’on possède à sa collection.

Elle permet également de suivre sa progression grâce à des statistiques et de personnaliser son profil.

## [SLIDE 6 : PUBLIC CIBLE]

Full7 s'adresse principalement aux joueurs et aux collectionneurs de jeux rétro. L'application est accessible depuis un navigateur, aussi bien sur ordinateur que sur mobile, sans avoir besoin d'installer quoi que ce soit.

## [SLIDE 7 : UTILISATEURS]

Il existe deux types d'utilisateurs. D'abord, les visiteurs, qui peuvent uniquement accéder aux pages de connexion et d'inscription. Ensuite, les utilisateurs connectés, qui peuvent parcourir le catalogue, gérer leur collection, consulter leurs statistiques et personnaliser leur profil.

Pour cette version, j'ai fait le choix de ne pas intégrer de compte administrateur. Le catalogue est directement enregistré dans la base de données et ne peut pas être modifié par les utilisateurs.

## [SLIDE 8 : CHOIX TECHNIQUES]

Pour développer Full7, j'ai utilisé PHP pour la partie serveur, MariaDB pour la base de données, et HTML, CSS et JavaScript pour l'interface. J'ai utilisé PDO pour communiquer avec la base de données et des requêtes préparées pour sécuriser les échanges. Enfin, j'ai travaillé en local avec XAMPP et utilisé Git pour suivre les modifications du code et travailler sur 2 poste.

## [SLIDE 9 : ARCHITECTURE]

Pour l'architecture, j'ai organisé mon projet en plusieurs dossiers afin de séparer les pages, les styles, les scripts et les éléments réutilisables.

J'ai notamment centralisé la connexion à la base de données et les éléments communs comme l'en-tête et le pied de page. J'ai également séparé les identifiants de connexion dans un fichier de configuration.

## [SLIDE 10 : MODÈLE MVC]

Concernant les architectures web, il existe notamment le modèle MVC, qui signifie Modèle, Vue, Contrôleur. Le modèle s'occupe des données, la vue de l'affichage et le contrôleur de la logique qui fait le lien entre les deux.

Je n’ai pas utilisé cette architecture dans Full7. Mon code PHP regroupe encore plusieurs responsabilités dans les pages. Le MVC aurait permis de mieux séparer ces responsabilités et de faciliter la maintenance du projet.

## [SLIDE 11 : CONCEPTION DE LA BASE DE DONNÉES, LES TABLES]

Pour concevoir la base de données, j’ai utilisé quatre tables principales.

La table consoles contient les informations sur les consoles.

La table games contient les jeux, chacun étant associé à une console.

La table users contient les comptes des utilisateurs.

Enfin, la table ownerships permet de faire le lien entre les utilisateurs et les jeux qu’ils possèdent.

Le catalogue contient dix consoles et cent jeux.

## [SLIDE 12 : CONCEPTION DE LA BASE DE DONNÉES, LES RELATIONS]

Cette dernière table est importante, car elle permet de gérer une relation plusieurs-à-plusieurs : un utilisateur peut posséder plusieurs jeux, et un même jeu peut être présent dans les collections de plusieurs utilisateurs.

## [SLIDE 13 : CONCEPTION DE LA BASE DE DONNÉES, LA COHÉRENCE DES DONNÉES]

J’ai également mis en place des contraintes pour garantir la cohérence des données. Par exemple, un utilisateur ne peut pas ajouter deux fois le même jeu à sa collection. Les suppressions en cascade permettent aussi d’éviter de conserver des associations qui ne correspondent plus à un utilisateur ou à un jeu existant.

Enfin, les mots de passe sont destinés à être stockés sous forme de hash, et non en clair. J’ai également prévu un script SQL pour recréer la base de données et importer les données.

## [SLIDE 14 : AUTHENTIFICATION ET SESSIONS]

J’ai commencé par mettre en place le système d’authentification. Les mots de passe ne sont jamais stockés en clair : ils sont hachés lors de l’inscription, puis vérifiés lors de la connexion.

J’utilise également les sessions PHP pour identifier les utilisateurs connectés et protéger les pages privées. Enfin, les requêtes préparées permettent de limiter les risques d’injections SQL.

## [SLIDE 15 : STATISTIQUES ET PROGRESSION]

Le tableau de bord permet à chaque utilisateur de suivre l'évolution de sa collection. Les statistiques sont calculées automatiquement à partir des jeux qu'il possède.

Pour cela, j'ai utilisé des requêtes SQL avec des jointures, des regroupements et une sous-requête. J'ai aussi prévu les cas particuliers, notamment lorsqu'un utilisateur ne possède encore aucun jeu, afin d'éviter les erreurs dans les calculs.

## [SLIDE 16 : GESTION DE LA COLLECTION]

La fonctionnalité principale de Full7 est la gestion de la collection. Depuis la fiche d'un jeu, l'utilisateur peut l'ajouter ou le retirer de sa collection.

J'ai également développé un système qui compare le nombre de jeux possédés au nombre total de jeux disponibles pour chaque console. Lorsque la collection est complète, un badge Full7 apparaît.

Pour obtenir ces résultats, j'utilise notamment un LEFT JOIN, qui me permet de conserver les jeux du catalogue même lorsqu'ils ne sont pas encore possédés par l'utilisateur.

## [SLIDE 17 : PERSONNALISATION DU PROFIL]

L'utilisateur peut également personnaliser son profil en modifiant son pseudo et sa photo de profil. Lorsqu'une photo est envoyée, son format est vérifié et le fichier est renommé pour éviter les collisions. Si aucune photo n'est ajoutée, l'image par défaut est conservée.

## [SLIDE 18 : SÉCURITÉ ET PROTECTION DES DONNÉES]

Pour la sécurité, j'ai identifié plusieurs risques et mis en place une mesure pour chacun.

Les mots de passe sont hachés avec password_hash(), et les injections SQL sont limitées grâce aux requêtes préparées PDO. Pour éviter les failles XSS, j'échappe les données affichées avec htmlspecialchars(). L'accès aux pages internes est protégé par une vérification de session, avec une redirection vers la connexion si l'utilisateur n'est pas connecté.

Pour les images envoyées, j'utilise une liste blanche d'extensions, un contrôle avec getimagesize() et un renommage du fichier. Enfin, les identifiants de la base de données sont dans un fichier secrets.php placé hors du dépôt Git, et les identifiants passés dans l'URL sont convertis en entier.

## [SLIDE 19 : PROTECTION DES DONNÉES PERSONNELLES (RGPD)]

Full7 ne collecte que le strict nécessaire au fonctionnement du compte : un pseudo, un email, un mot de passe, une photo de profil facultative et la liste des jeux possédés.

L'email n'est jamais affiché aux autres utilisateurs, et la collection n'est visible que par son propriétaire. Je n'utilise aucune donnée sensible, aucune géolocalisation et aucun outil de suivi publicitaire. Le seul cookie est celui de la session PHP.

Une limite actuelle : l'utilisateur peut modifier son pseudo et sa photo, mais pas supprimer son compte lui-même. C'est une évolution prévue pour respecter le droit à l'effacement. Pour l'instant, la suppression d'un utilisateur supprime déjà automatiquement sa collection grâce au ON DELETE CASCADE.

## [SLIDE 20 : CONCLUSION]

Pour conclure, Full7 répond à l'objectif de départ : permettre à un collectionneur de rétro gaming de gérer sa collection et de suivre sa progression, depuis un simple navigateur.

Ce projet m'a permis de concevoir une base de données relationnelle, de mettre en place l'authentification avec des mots de passe hachés, des sessions et des requêtes préparées, et de structurer une application web en PHP.

Il reste plusieurs pistes d'amélioration : passer à une architecture MVC pour mieux séparer les responsabilités, ajouter un compte administrateur pour gérer le catalogue, permettre à l'utilisateur de supprimer son compte, et élargir le nombre de consoles et de jeux grâce à une api.

Je vous remercie de votre attention, et je suis disponible pour répondre à vos questions.